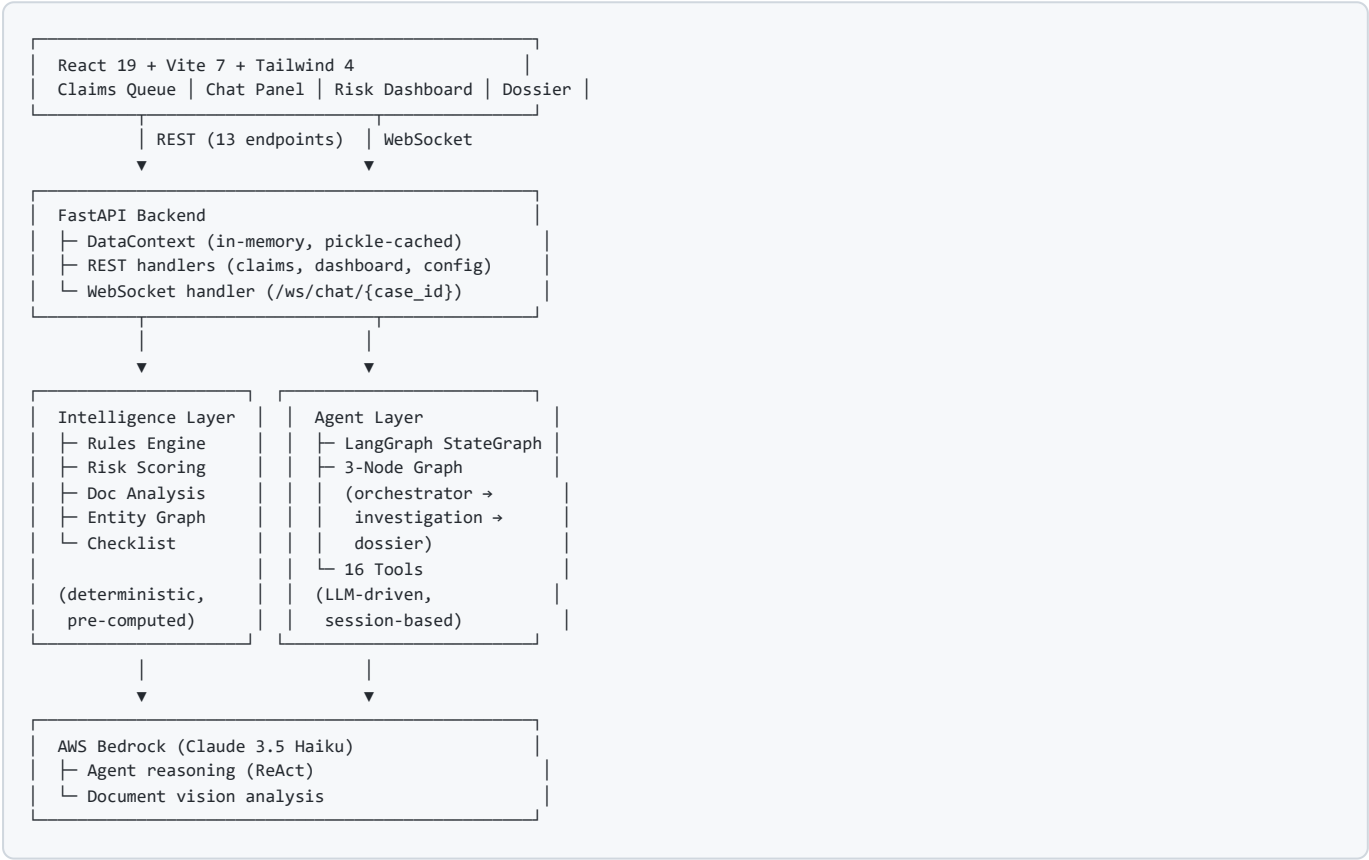


Claims Copilot — Architecture & Design Decisions

Why This Architecture

This system is built around one core principle: **augment the examiner, don't replace them**. Every architectural decision flows from that — the AI surfaces patterns and evidence, but the human makes the call.

System Overview



Key Architecture Decisions

1. Deterministic Rules First, ML Second, LLM Third

Decision: The intelligence pipeline runs in three layers — and the LLM is the *last* layer, not the first.

Why:

- In fraud, you need **defensible findings**. "Rule R-001 triggered because member has 35 dependents" is auditable. "The AI thinks this looks suspicious" is not.
- Rules Engine (15 rules) catches the obvious fraud — these are the patterns real SIU teams already look for.
- Risk Scoring (hybrid model) ranks everything else by suspicion level so examiners prioritize correctly.
- The LLM copilot helps the examiner *investigate* what the deterministic layers flagged. It doesn't do the flagging.

Implication: If you turned off Bedrock entirely, the system still works. The rules still fire, claims still get scored, the queue still prioritizes. The LLM makes it faster and more conversational — it doesn't make it functional.

2. LangGraph 3-Node State Machine (Not a Simple Chain)

Decision: The agent uses a LangGraph `StateGraph` with three specialized nodes, not a single ReAct loop.



State tracked across iterations:

```

class InvestigationState(TypedDict):
    messages: Sequence[BaseMessage] # Full conversation history
    current_phase: str                # scan → investigate → compile → done
    findings: dict                    # Accumulated evidence
    dossier: str                      # Generated report
    evidence_sufficient: bool         # Gate for dossier compilation
    loop_count: int                  # Prevents infinite loops
    compile_loop_count: int          # Tracks dossier rejection cycles
  
```

Why not a single agent with all 16 tools?

- Investigation tools and dossier tools have different failure modes. Investigation tools *gather* — they should run speculatively. Dossier tools *conclude* — they should only run when evidence is sufficient.
- The orchestrator node acts as a **quality gate**: it checks `evidence_sufficient` before routing to the dossier node. If the dossier node rejects (insufficient evidence), it routes *back* to investigation for deeper analysis.
- This creates a natural **self-healing loop**: investigate → assess → reject → investigate deeper → assess again → accept → compile.

Why this matters for production:

- Each node can be independently tested, monitored, and rate-limited.
- The orchestrator's routing logic is deterministic — you can trace *why* the agent went to investigation vs. dossier at any step.
- Loop counts prevent runaway API costs.

3. Tool-Based Architecture with Context Injection

Decision: Agent capabilities are exposed as 16 discrete tools with a shared `DataContext`, not as monolithic prompt engineering.

Tool pattern:

```
def check_eligibility(
    claim_id: Annotated[str, "Claim ID (e.g. WC-247)"],
) -> Dict:
    """Check claim eligibility: policy active, coverage matches, member enrolled."""
    claim = _context.get_claim(claim_id)
    policy = _context.get_policy(claim.policy_id)
    # ... deterministic logic ...
    return {"claim_id": claim_id, "eligible": True, "issues": []}
```

Why:

- The LLM decides *which* tools to call and *when*, but the tools themselves are deterministic Python. This means:
 - Tool outputs are reproducible and testable
 - The LLM can't hallucinate data — it can only work with what the tools return
 - Each tool call is logged (`_tool_log`), creating an audit trail
- Tools are grouped by concern:
 - **7 Fraud tools** — eligibility, family claims, documents, dependents, provider patterns, inconsistencies, related claims
 - **5 Dossier tools** — evidence assessment, regulatory search, similar cases, recovery estimation, dossier compilation
 - **4 Policy/Workflow tools** — policy details, state rules, alerts, coverage matching

Context injection:

```
_context: DataContext = None

def set_context(ctx: DataContext):
    global _context
    _context = ctx
```

This keeps tools stateless (any tool can run independently) while sharing a single data source. In production, this would be replaced with database connections — the tool signatures wouldn't change.

4. Hybrid Risk Scoring: Rules-Dominant by Design

Decision: Rules contribute 35% of the score weight — more than any single feature category.

Scoring formula:

```
Final Score = (Feature Score × 0.4) + (Rules Boost × up to 60 points)

Feature Score (0-50 points):
├ Policy features      (10%) – policy age, recent ownership/beneficiary changes
├ Member features     (15%) – tenure, dependent count, termination proximity
├ Claim features      (15%) – amount ratio, resubmission flag, contact count
├ Provider features   (10%) – claim volume, no-facility indicator
├ Network features    (10%) – shared address count, family claim density
└ Temporal features   ( 5%) – service-to-filing gap, recent claim frequency

Rules Boost (0-60 points):
├ Each BLOCK rule:    +0.5 (normalized) → up to 30 points
├ Each FLAG rule:     +0.25           → up to 15 points
└ Each INFO rule:     +0.05           → up to 3 points

Tiers: HIGH (≥60) | MEDIUM (30-59) | LOW (<30)
```

Why rules-dominant?

- In supplemental health, the fraud patterns are well-known (dependent abuse, provider mills, termination rush). You don't need ML to find them — you need rules. ML helps with the gray area.
- Making rules the primary score driver means the scoring is explainable. An examiner can ask "why is this HIGH?" and get "R-001 triggered: member has 35 dependents" — not "the model's internal weights."
- The ML features (Isolation Forest anomaly detection, feature-based scoring) catch *novel* patterns the rules don't cover — but they're weighted lower because they're harder to explain.

5. Entity Graph for Network Detection

Decision: Build a NetworkX directed graph of all entities and their relationships, then run pattern detection algorithms on it.

Graph structure:

```
Member —EMPLOYED_BY→ Employer
Member —LIVES_AT→ Address
Member —ENROLLED_IN→ Policy
Dependent —DEPENDENT_OF→ Member
Claim —FILED_BY→ Member
Claim —TREATED_BY→ Provider
Claim —SERVICED_AT→ Facility
Provider —PRACTICES_AT→ Facility
```

5 detected pattern types:

1. **Dependent Ring** — 3+ members at shared address with overlapping dependents (severity: HIGH if >10 dependents)
2. **Provider Cluster** — Single provider with >30 claims (HIGH if >50)
3. **Shared Address Group** — Multiple unrelated members at same address
4. **Termination Rush** — 2+ claims within 14 days of employee termination
5. **Document Tampering Cluster** — Claims with correlated document integrity failures

Why a graph?

- Fraud rings are inherently relational. You can't detect "4 members at the same address filing through the same provider" by looking at individual claims — you need the connections.
- NetworkX is lightweight (no database needed) and the graph fits in memory for 500 claims. In production, this would move to Neo4j or Amazon Neptune.

6. Pre-Computed Intelligence, Live Copilot

Decision: Rules, risk scores, document checks, and graph analysis are computed at startup and cached. The LLM copilot queries these results at runtime — it doesn't re-compute them.

Startup pipeline (DataContext initialization):

```
generate_supplemental_data()      # Synthetic data
→ run_rules_engine(claim, ctx)    # 15 rules × 500 claims
→ score_claim(claim, ctx)         # Risk scoring × 500 claims
→ run_document_checks(doc)        # 13 checks × documents
→ build_supplemental_health_graph()# Entity graph
→ detect_patterns(graph, data)    # Pattern detection
→ build_case_queue(...)           # Priority queue
→ pickle cache to disk
```

Why pre-compute?

- Rules and scoring are deterministic — no reason to recompute per request.
- First load takes ~2-3 seconds. Subsequent loads from pickle cache are instant.
- The copilot session calls `_context.get_claim()`, `_context.claim_rules[claim_id]`, etc. — fast dictionary lookups, not expensive recalculations.
- In production, this would be a batch pipeline (nightly or on-claim-intake) feeding a database.

7. WebSocket for Real-Time Copilot Chat

Decision: The copilot chat uses WebSocket (`/ws/chat/{case_id}`), not REST polling.

```
@app.websocket("/ws/chat/{case_id}")
async def websocket_endpoint(websocket: WebSocket, case_id: str):
    await websocket.accept()
    session = get_or_create_session(case_id, ...)
    while True:
        message = await websocket.receive_text()
        response = handle_analyst_message_sync(case_id, message)
        await websocket.send_json({"response": response})
```

Why:

- LLM tool-calling takes 3-10 seconds. With REST, the frontend would need to poll. With WebSocket, the response streams back as soon as it's ready.
- One WebSocket connection per claim session = natural session lifecycle. When the examiner navigates away, the connection closes.

8. Session-Per-Claim with Message History

Decision: Each claim gets an independent `CopilotSession` with full message history.

```
@dataclass
class CopilotSession:
    session_id: str
    case_id: str
    claim_type: str
    subject_id: str
    messages: List[BaseMessage] # Full LangChain message history
    tools_called: List[str] # Audit trail of tool invocations
    checklist_state: Dict # Step completion tracking
```

Why:

- An examiner working WC-247 shouldn't see conversation context from WC-112.
- Message history lets the copilot reference earlier findings: "As I mentioned, R-001 triggered for this member" — without re-calling tools.
- `tools_called` tracks which tools were invoked per session, useful for audit and debugging.

9. 7-Step Checklist as Structured Investigation

Decision: Every claim investigation follows a fixed 7-step checklist, with each step producing a pass/fail/needs_review result.

Step	Name	Auto-Passable?	What It Checks
1	Initial Review	Yes	Required fields present, task assigned
2	Eligibility Verification	Yes	Policy active, coverage type matches
3	Fraud Screening	Conditional	Risk score, BLOCK rules, suspicious banner
4	Family & Network Check	Yes	R-001/R-010/R-015 not triggered
5	Medical Documentation	Yes	13 document checks pass
6	Policy & Coverage	Yes	Amount within limits, dates in window
7	Final Determination	Never	Examiner decides: approve/deny/escalate

Why a fixed checklist?

- Insurance regulators (state DOI) audit claim handling procedures. A checklist proves the examiner followed a consistent process.
- Auto-passable steps mean low-risk claims move through steps 1-6 automatically — the examiner only intervenes at step 7 or when a step fails.
- This is the real productivity gain: an examiner handling 30-50 claims/day doesn't need to manually verify eligibility on every single one.

10. Dossier Self-Healing Loop

Decision: The dossier node can *reject its own output* and send the investigation back for more evidence.

Flow:

```
Investigation gathers evidence
→ Orchestrator routes to Dossier
→ assess_evidence() runs 6 sufficiency checks:
  ✓ ≥3 independent evidence points
  ✓ Statistical significance (z-score >2 OR risk >60)
  ✓ Temporal analysis present
  ✓ Network evidence (for ring patterns)
  ✓ Regulatory citation
  ✓ Document integrity (for tampering)
→ If INSUFFICIENT: orchestrator routes BACK to Investigation
→ Investigation runs deeper tools (provider patterns, related claims)
→ Back to Dossier → assess again
→ If SUFFICIENT: compile_dossier() generates markdown report
```

Why:

- Prevents the LLM from generating a dossier with weak evidence. The sufficiency check is deterministic — 6 concrete criteria, not "does the LLM think it's enough."
- In demo mode, this creates a compelling narrative: the system tries to compile a dossier, rejects itself, investigates deeper, and eventually produces a thorough report. This mirrors how real SIU analysts work — they don't write the referral after the first look.

Technology Choices

Choice	Why
LangGraph over LangChain AgentExecutor	StateGraph gives explicit control over routing, loop limits, and phase transitions. AgentExecutor is a black box.
FastAPI over Flask/Django	Native async, WebSocket support, auto-generated OpenAPI docs. Minimal boilerplate.
Claude 3.5 Haiku over Sonnet/Opus	Haiku is 10x cheaper and fast enough for tool-calling. Fraud investigation doesn't need creative writing — it needs structured reasoning.
NetworkX over Neo4j	500 claims fit in memory. No database setup needed for a demo. Same algorithms, simpler deployment.
Pickle caching over database	Demo-appropriate. First load generates + caches. Subsequent loads are instant. In production, this would be PostgreSQL + Redis.
React 19 + Vite 7	Fast dev iteration, component-based UI. Vite's HMR means frontend changes appear instantly during development.
Tailwind 4	No custom CSS to maintain. Utility classes keep the UI consistent without a design system.

What Would Change for Production

Demo	Production
Synthetic 500 claims	EDI/837 claim ingestion pipeline
Pickle cache	PostgreSQL + Redis
In-memory sessions	Redis-backed sessions with TTL
NetworkX	Neo4j or Amazon Neptune
Single-user	RBAC, SSO, audit logging
<code>_context</code> global	Dependency injection / request-scoped
No encryption	HIPAA-compliant storage, TLS everywhere
Claude Haiku only	Model routing (Haiku for tools, Sonnet for dossier narrative)
Document metadata checks	Real OCR + pixel-level tamper detection